Problema 2SAT

Aurélio Gabriel Ortiz Vieira
GRR20231953
Universidade Federal do Paraná – UFPR

I. *Exercício 3.28*

$$(x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee \overline{x_3}) \wedge (x_1 \vee x_2) \wedge (\overline{x_3} \vee x_4) \wedge (\overline{x_1} \vee x_4)$$

a)

Tabela I
TABELA VERDADE PARA A CLÁUSULA DADA, A COLUNA *Todas as Cláusulas* REPRESENTA O RESULTADO DA INSTÂNCIA DADA AS CLÁUSULAS NA TABELA

x_1	x_2	x_3	x_4	$(x_1 \vee \overline{x_2})$	$(\overline{x_1} \vee \overline{x_3})$	$(x_1 \vee x_2)$	$(\overline{x_3} \vee x_4)$	$(\overline{x_1} \vee x_4)$	Todas as Cláusulas
0	0	0	0	1	1	0	1	1	0
0	0	0	1	1	1	0	1	1	0
0	0	1	0	1	1	0	0	1	0
0	0	1	1	1	1	0	1	1	0
0	1	0	0	0	1	1	1	1	0
0	1	0	1	0	1	1	1	1	0
0	1	1	0	0	1	1	0	1	0
0	1	1	1	0	1	1	1	1	0
1	0	0	0	1	1	1	1	0	0
1	0	0	1	1	1	1	1	1	1
1	0	1	0	1	0	1	0	0	0
1	0	1	1	1	0	1	1	1	0
1	1	0	0	1	1	1	1	0	0
1	1	0	1	1	1	1	1	1	1
1	1	1	0	1	0	1	0	0	0
1	1	1	1	1	0	1	1	1	0

Conforme mostra a tabela I, há mais uma atribuição verdadeira para o problema 2SAT: *verdadeiro, verdadeiro, falso, verdadeiro*.

b)

Para encontrar uma instância 2SAT de quatro variáveis que nunca seja satisfeita, basta encontrar uma instância com 1 variável que não seja satisfeita, uma vez que se apenas 2 cláusulas (independente da quantidade de variáveis) nunca puderem ser verdadeiras simultaneamente a instância nunca será verdadeira. A tabela II lista todas as cláusulas possíveis para 1 variável.

Tabela II
TABELA VERDADE DE TODAS AS CLÁUSULAS POSSÍVEIS PARA UMA VARIÁVEL

x_1	$(x_1 \vee x_1)$	$(x_1 \vee \overline{x_1})$	$(\overline{x_1} \vee \overline{x_1})$	Todas as Cláusulas
0	0	1	1	0
1	1	1	0	0

Como $(x_1 \vee x_1)$ é equivalente a x_1, e $(\overline{x_1} \vee \overline{x_1})$ é equivalente a $\overline{x_1}$, essas duas cláusulas nunca poderão ser satisfeitas simultaneamente em uma instância 2SAT, já que uma é a negação da outra.
Dessa forma, uma instância possível de 2SAT com 4 variáveis que nunca é satisfeita é:
$(x_1 \vee x_1) \wedge (\overline{x_1} \vee \overline{x_1}) \wedge (\overline{x_2} \vee \overline{x_1}) \wedge (x_3 \vee x_4)$

c)

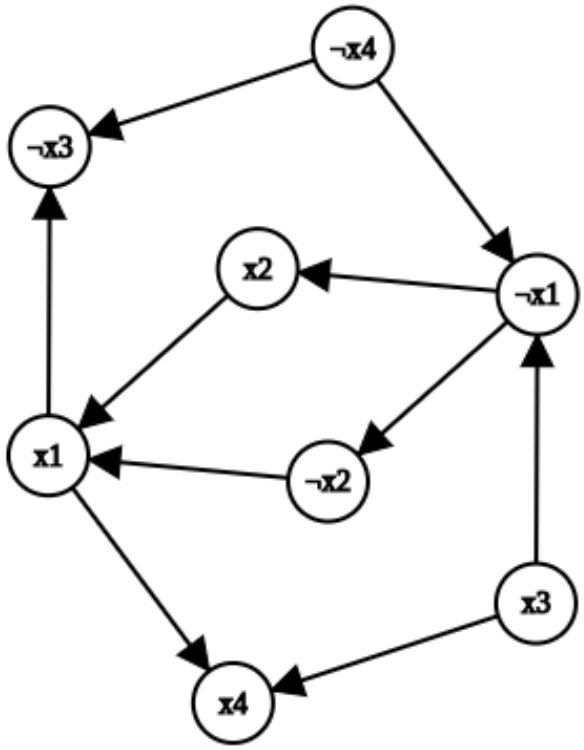


Figura 1. Representação em grafo da instância $(x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee \overline{x_3}) \wedge (x_1 \vee x_2) \wedge (\overline{x_3} \vee x_4) \wedge (\overline{x_1} \vee x_4)$

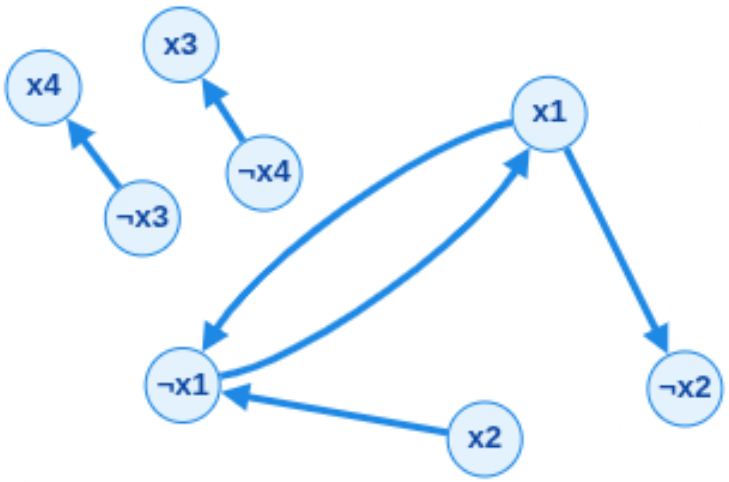


Figura 2. Representação em grafo da instância $(x_1 \vee x_1) \wedge (\overline{x_1} \vee \overline{x_1}) \wedge (\overline{x_2} \vee \overline{x_1}) \wedge (x_3 \vee x_4)$

d)

- Ter uma aresta de x_1 para $\overline{x_1}$ significa ter uma cláusula $(x_1 \vee x_1)$, que é equivalente a x_1, e ter uma aresta de $\overline{x_1}$ para x_1, significa ter uma cláusula $(\overline{x_1} \vee \overline{x_1})$, equivalente a $\overline{x_1}$ (como pode ser visto pela tabela III).
- Ter uma componente fortemente conexa entre x_1 e $\overline{x_1}$ significa que as cláusulas $(x_1 \vee x_1)$ e $(\overline{x_1} \vee \overline{x_1})$ estão na mesma instância.
- Uma vez que se x_1 = 0 $\overline{x_1}$ = 1, ambas as cláusulas nunca poderão ser satisfeitas ao mesmo tempo, tornando a instância insatisfatível.

Tabela III
TABELA VERDADE QUE DEMONSTRA EQUIVALÊNCIA ENTRE $(x_1 \vee x_1)$ E x_1, E ENTRE $(\overline{x_1} \vee \overline{x_1})$ E $\overline{x_1}$.

x_1	$(x_1 \vee x_1)$	$\overline{x1}$	$(\overline{x_1} \vee \overline{x_1})$
0	0	1	1
1	1	0	0

e)

Algoritmo 1 Encontra_atribuição_verdadeira_de_G_i

dado: G_i finito

1: **para todo:** CFC (Componente Fortemente Conexa) sorvedouro $\in G_i$ **faça:**
2: **para todo:** Literal l $\in$ CFC **faça:**
l = V
$\overline{l}$ = F
3: **fim para**
4: **fim para**

Tabela IV
TABELA VERDADE DA IMPLICAÇÃO, EM NEGRITO O RESULTADO DA IMPLICAÇÃO SEMPRE QUE $x_1 = V$

x_1	x_2	$\overline{x_2} \Rightarrow x_1$
0	0	0
0	1	1
1	0	**1**
1	1	**1**

Prova I

- $(x_1 \vee x_2) \equiv \overline{x_2} \Rightarrow x_1$
- Pela tabela IV, sempre que $x_1 = V$ a implicação $(\overline{x_2} \Rightarrow x_1)$ será também sempre V
- Seja x_1 um CFC sorvedouro $\in G_i$
- Aplicando o algoritmo 1 sempre encontraremos uma atribuição verdadeira, em G_i desde que este não contenha CFC com um literal e sua negação.

f)

Para criar um algoritmo linear que resolva 2SAT basta adicionar ao algoritmo 1 uma forma de verificar se o algoritmo pode ser satisfeito, e também adicionar o algoritmo de Tarjan que encontra as CFCs com custo linear [1]:

Algoritmo 2 Resolve_2SAT

dado: G_i finito
1: $lista_CFC \leftarrow$ Tarjan(G)
2: **para todo:** variável x_i **faça:**
3: **se** $CFC(x_i) = CFC(\overline{x_i})$ **então**
4: **retorne** Insatisfatível
5: **fim se**
6: **fim para**
7: Inicialize todos os literais como não atribuídos
8: **para todo:** CFC (Componente Fortemente Conexa) $\in$ lista_CFC **faça:**
9: **para todo:** Literal l $\in$ CFC **faça:**
l = V
$\bar{l}$ = F
10: **fim para**
11: **fim para**
12: **retorne** Atribuição encontrada

O algoritmo de Tarjan possui custo $O(|V| + |E|)$, a verificação de satisfação (feita conferindo se há algum literal e sua negação em uma mesma CFC) tem custo $O(|V|/2)$, e o loop de atribuição de verdadeiro ou falso aos literais tem custo $O(|V|)$ (pois passa por todos literais em cada CFC). Sendo assim, o **custo total para Resolve_2SAT é:** $O(3|V| + |E|)$.

REFERÊNCIAS

[1] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.